

Module M214

Application Cloud Native

Séance 3 : Les Middlewares

Express.js

Middlewares & Authentification JWT

OFPPT

Développement Digital - Option Full Stack

Table des matières

1	Introduction aux Middlewares	2
1.1	Qu'est-ce qu'un Middleware ?	2
1.2	Le Pipeline de Middlewares	2
1.3	Que peut faire un Middleware ?	2
1.4	Syntaxe de base	3
2	Middlewares Intégrés (Built-in)	3
2.1	express.json()	3
2.2	express.urlencoded()	3
2.3	express.static()	4
3	Middlewares Personnalisés	5
3.1	Middleware de Logging	5
3.2	Middleware de Validation	5
3.3	Middleware Factory	6
4	Authentification avec JWT	7
4.1	Qu'est-ce que JWT ?	7
4.2	Pourquoi JWT ?	7
4.3	Structure d'un JWT	7
4.3.1	1. Header (En-tête)	7
4.3.2	2. Payload (Charge utile)	8
4.3.3	3. Signature	8
4.4	Flux d'authentification JWT	8
4.5	Installation	9
4.6	Générer un Token (Login)	9
4.7	Route d'inscription	10
4.8	Middleware de vérification JWT	10
4.9	Protéger les routes	11
4.10	Utilisation côté Client	12
5	Gestion des Erreurs	13
5.1	Middleware d'erreur	13
5.2	Déclencher une erreur	13
5.3	Middleware 404	13
6	Ordre d'Exécution	14
7	Middlewares Tiers	15
7.1	CORS	15
7.2	Morgan	15
7.3	Autres middlewares utiles	15
8	Application Complète	16
9	Synthèse	18
9.1	Récapitulatif JWT	18
9.2	Template Express + JWT	18

1 Introduction aux Middlewares

1.1 Qu'est-ce qu'un Middleware ?

Middleware

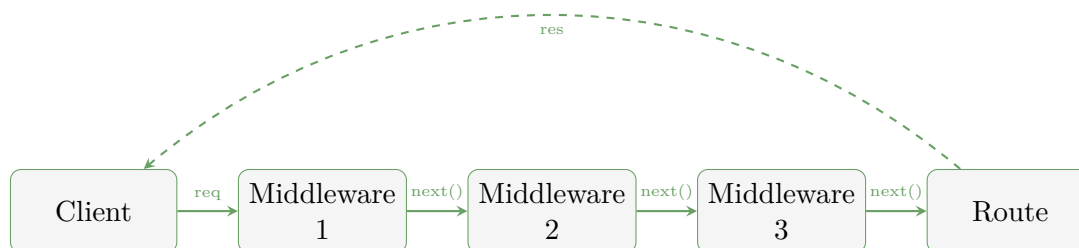
Un **middleware** est une fonction qui s'exécute pendant le cycle de vie d'une requête HTTP, entre le moment où le serveur reçoit la requête et le moment où il envoie la réponse.

Un middleware a accès à trois éléments :

- L'objet **request** (**req**) - informations de la requête
- L'objet **response** (**res**) - pour envoyer une réponse
- La fonction **next** - passe au middleware suivant

1.2 Le Pipeline de Middlewares

Dans Express, chaque requête HTTP traverse une chaîne de middlewares avant d'atteindre la route finale.



1.3 Que peut faire un Middleware ?

1. **Exécuter du code** : logging, validation, authentification
2. **Modifier req et res** : ajouter des propriétés, modifier des headers
3. **Terminer le cycle** : envoyer une réponse (pas de **next()**)
4. **Passer au suivant** : appeler **next()**

⚠ Important

Si un middleware n'envoie pas de réponse, il **doit** appeler **next()**. Sinon, la requête reste bloquée indéfiniment.

1.4 Syntaxe de base

```
1 const monMiddleware = (req, res, next) => {
2   console.log('Middleware execute');
3
4   // Option 1: Passer au suivant
5   next();
6
7   // Option 2: Terminer avec une reponse
8   // res.json({ message: 'Termine ici' });
9 };
10
11 // Utilisation globale
12 app.use(monMiddleware);
13
14 // Utilisation sur un chemin
15 app.use('/api', monMiddleware);
16
17 // Utilisation sur une route
18 app.get('/users', monMiddleware, (req, res) => { ... });
```

Listing 1 – Structure d'un middleware

2 Middlewares Intégrés (Built-in)

Express fournit des middlewares essentiels prêts à l'emploi.

2.1 express.json()

Parse les requêtes avec un corps JSON et expose le résultat dans `req.body`.

```
1 const express = require('express');
2 const app = express();
3
4 // OBLIGATOIRE pour recevoir du JSON
5 app.use(express.json());
6
7 app.post('/api/users', (req, res) => {
8   console.log(req.body); // { nom: 'Ahmed', email: 'ahmed@mail
9   .com' }
10   res.status(201).json(req.body);
11 });
```

Listing 2 – Middleware JSON - Obligatoire pour les APIs

✖ Sans `express.json()`

Si vous oubliez `express.json()`, `req.body` sera `undefined` pour toutes vos routes POST/PUT!

2.2 express.urlencoded()

Parse les données de formulaires HTML (`application/x-www-form-urlencoded`).

```
1 app.use(express.urlencoded({ extended: true }));
2 // extended: true permet les objets imbriqués
```

Listing 3 – Middleware pour formulaires HTML

2.3 express.static()

Sert des fichiers statiques (HTML, CSS, JS, images).

```
1 // Servir les fichiers du dossier 'public'
2 app.use(express.static('public'));
3
4 // Structure:
5 // public/
6 //   |-- index.html      -> http://localhost:3000/index.html
7 //   |-- css/style.css   -> http://localhost:3000/css/style.css
8
9 // Avec prefixe virtuel
10 app.use('/static', express.static('public'));
11 // -> http://localhost:3000/static/index.html
```

Listing 4 – Servir des fichiers statiques

3 Middlewares Personnalisés

3.1 Middleware de Logging

Enregistre les informations de chaque requête.

```
1 const logger = (req, res, next) => {
2   const timestamp = new Date().toISOString();
3   console.log(`[${timestamp}] ${req.method} ${req.url}`);
4   next();
5 };
6
7 app.use(logger);
8 // Resultat: [2024-01-15T10:30:00.000Z] GET /api/users
```

Listing 5 – Logger personnalisé

3.2 Middleware de Validation

Vérifie les données avant traitement.

```
1 const validateUser = (req, res, next) => {
2   const { nom, email } = req.body;
3   const errors = [];
4
5   if (!nom || nom.trim().length < 2) {
6     errors.push('Nom invalide (min 2 caracteres)');
7   }
8
9   if (!email || !email.includes('@')) {
10    errors.push('Email invalide');
11  }
12
13  if (errors.length > 0) {
14    return res.status(400).json({ success: false, errors });
15  }
16
17  next();
18 };
19
20 app.post('/api/users', validateUser, (req, res) => {
21   // Donnees validees
22   res.status(201).json({ message: 'User cree' });
23 });
```

Listing 6 – Validation des données

3.3 Middleware Factory

Fonction qui retourne un middleware configurable.

```
1 const validateFields = (requiredFields) => {
2   return (req, res, next) => {
3     const missing = requiredFields.filter(f => !req.body[f]);
4
5     if (missing.length > 0) {
6       return res.status(400).json({
7         error: 'Champs manquants: ${missing.join(', ')}';
8       });
9     }
10    next();
11  };
12 };
13
14 // Utilisation
15 app.post('/api/users', validateFields(['nom', 'email']), ...);
16 app.post('/api/products', validateFields(['nom', 'prix']), ...);
```

Listing 7 – Middleware réutilisable

4 Authentification avec JWT

4.1 Qu'est-ce que JWT ?

JSON Web Token (JWT)

JWT est un standard ouvert (RFC 7519) qui définit un moyen compact et autonome de transmettre des informations de manière sécurisée entre deux parties sous forme d'un objet JSON.

Cas d'utilisation principal : L'authentification. Après la connexion, chaque requête inclut le JWT, permettant à l'utilisateur d'accéder aux ressources autorisées.

4.2 Pourquoi JWT ?

Avantage	Description
Stateless	Le serveur n'a pas besoin de stocker les sessions en mémoire
Scalable	Facilite la mise à l'échelle (plusieurs serveurs)
Mobile-friendly	Idéal pour les APIs consommées par des apps mobiles
Cross-domain	Fonctionne facilement avec CORS

4.3 Structure d'un JWT

Un JWT est composé de **trois parties** séparées par des points :

HEADER.PAYLOAD.SIGNATURE

```
1 eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.  
2 eyJpZCI6IjEyMyIsIm5vbiI6IkpobWVkaWiaWF0IjoxNjE2MjM5MDIyfQ.  
3 SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```

Listing 8 – Exemple de JWT

4.3.1 1. Header (En-tête)

Contient le type de token et l'algorithme de signature.

```
1 {  
2   "alg": "HS256",      // Algorithme: HMAC SHA-256  
3   "typ": "JWT"         // Type: JWT  
4 }
```

Listing 9 – Header décodé

4.3.2 2. Payload (Charge utile)

Contient les **claims** (déclarations) - les données de l'utilisateur.

```
1 {  
2   "id": "123",           // Identifiant utilisateur  
3   "nom": "Ahmed",       // Donnees personnalisées  
4   "role": "admin",      // Role de l'utilisateur  
5   "iat": 1616239022,    // Issued At: date de creation  
6   "exp": 1616325422     // Expiration: date d'expiration  
7 }
```

Listing 10 – Payload décodé

⚠ Sécurité

Le payload est encodé en Base64, pas chiffré ! N'y mettez **jamais** de données sensibles (mots de passe, numéros de carte, etc.).

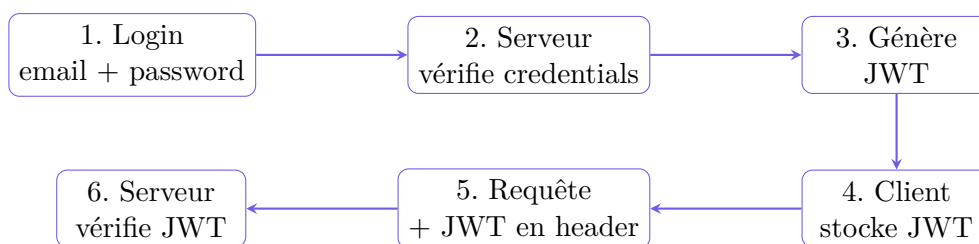
4.3.3 3. Signature

Garantit que le token n'a pas été modifié.

```
1 HMACSHA256(  
2   base64UrlEncode(header) + "." + base64UrlEncode(payload),  
3   SECRET_KEY  
4 )
```

Listing 11 – Calcul de la signature

4.4 Flux d'authentification JWT



4.5 Installation

```
1 npm install jsonwebtoken bcryptjs
```

- jsonwebtoken : Créer et vérifier les tokens JWT
- bcryptjs : Hasher les mots de passe

4.6 Générer un Token (Login)

```
1 const jwt = require('jsonwebtoken');
2 const bcrypt = require('bcryptjs');
3 const User = require('./models/User');
4
5 // Cle secrete (en production: variable d'environnement)
6 const JWT_SECRET = 'ma_cle_secrete_tres_longue_123!';
7 const JWT_EXPIRES_IN = '24h'; // Duree de validite
8
9 app.post('/api/auth/login', async (req, res) => {
10   try {
11     const { email, password } = req.body;
12
13     // 1. Verifier si l'utilisateur existe
14     const user = await User.findOne({ email });
15     if (!user) {
16       return res.status(401).json({ error: 'Email ou mot de
17         passe incorrect' });
18     }
19
20     // 2. Verifier le mot de passe
21     const isMatch = await bcrypt.compare(password, user.
22       password);
23     if (!isMatch) {
24       return res.status(401).json({ error: 'Email ou mot de
25         passe incorrect' });
26     }
27
28     // 3. Generer le token JWT
29     const token = jwt.sign(
30       { id: user._id, email: user.email, role: user.role },
31       // Payload
32       JWT_SECRET,
33       // Cle secrete
34       { expiresIn: JWT_EXPIRES_IN }
35       // Options
36     );
37
38     // 4. Envoyer le token au client
39     res.json({
40       success: true,
41       token,
42       user: { id: user._id, nom: user.nom, email: user.
43         email }
```

```
37     });
38
39     } catch (error) {
40         res.status(500).json({ error: error.message });
41     }
42 });
```

Listing 12 – Route de connexion avec JWT

4.7 Route d'inscription

```
1 app.post('/api/auth/register', async (req, res) => {
2     try {
3         const { nom, email, password } = req.body;
4
5         // Verifier si l'email existe deja
6         const existingUser = await User.findOne({ email });
7         if (existingUser) {
8             return res.status(400).json({ error: 'Email deja
utilise' });
9         }
10
11         // Hasher le mot de passe (10 = nombre de rounds)
12         const hashedPassword = await bcrypt.hash(password, 10);
13
14         // Creer l'utilisateur
15         const user = await User.create({
16             nom,
17             email,
18             password: hashedPassword
19         });
20
21         // Generer le token
22         const token = jwt.sign(
23             { id: user._id, email: user.email },
24             JWT_SECRET,
25             { expiresIn: JWT_EXPIRES_IN }
26         );
27
28         res.status(201).json({ success: true, token });
29
30     } catch (error) {
31         res.status(500).json({ error: error.message });
32     }
33 });
```

Listing 13 – Inscription avec hashage du mot de passe

4.8 Middleware de vérification JWT

```
1 const authenticateToken = (req, res, next) => {
2     // 1. Recuperer le header Authorization
```

```

3   const authHeader = req.headers['authorization'];
4   // Format attendu: "Bearer <token>"
5
6   if (!authHeader) {
7       return res.status(401).json({ error: 'Token manquant' });
8   }
9
10  // 2. Extraire le token
11  const parts = authHeader.split(' ');
12  if (parts.length !== 2 || parts[0] !== 'Bearer') {
13      return res.status(401).json({ error: 'Format: Bearer <
token>' });
14  }
15
16  const token = parts[1];
17
18  // 3. Verifier le token
19  try {
20      const decoded = jwt.verify(token, JWT_SECRET);
21
22      // 4. Ajouter les donnees utilisateur a req
23      req.user = decoded; // { id, email, role, iat, exp }
24
25      next(); // Token valide, continuer
26
27  } catch (error) {
28      if (error.name === 'TokenExpiredError') {
29          return res.status(401).json({ error: 'Token expire'
});
30      }
31      return res.status(401).json({ error: 'Token invalide' });
32  }
33 };
34
35 module.exports = authenticateToken;

```

Listing 14 – Middleware d'authentification JWT

4.9 Protéger les routes

```

1   const authenticateToken = require('./middlewares/auth');
2
3   // Route publique - pas d'authentification
4   app.get('/api/products', (req, res) => {
5       res.json({ products: [] });
6   });
7
8   // Route protegee - necessite un token valide
9   app.get('/api/profile', authenticateToken, (req, res) => {
10      // req.user contient les donnees du token
11      res.json({
12          message: 'Acces autorise',
13          user: req.user

```

```
14   });
15 });
16
17 // Route protegee avec role specifique
18 const authorizeRole = (...roles) => {
19   return (req, res, next) => {
20     if (!roles.includes(req.user.role)) {
21       return res.status(403).json({ error: 'Acces refuse'
22     });
23     }
24     next();
25   };
26 };
27
28 app.delete('/api/users/:id',
29   authenticateToken,           // Verifier le token
30   authorizeRole('admin'),       // Verifier le role
31   async (req, res) => {
32     // Seulement les admins peuvent supprimer
33     await User.findByIdAndDelete(req.params.id);
34     res.json({ message: 'Utilisateur supprime' });
35   });
```

Listing 15 – Routes protégées par JWT

4.10 Utilisation côté Client

```
1 // Apres login, stocker le token
2 localStorage.setItem('token', response.token);
3
4 // Pour chaque requete protegee, inclure le token
5 const token = localStorage.getItem('token');
6
7 fetch('/api/profile', {
8   method: 'GET',
9   headers: {
10     'Content-Type': 'application/json',
11     'Authorization': 'Bearer ${token}' // Header important!
12   }
13 })
14 .then(res => res.json())
15 .then(data => console.log(data));
```

Listing 16 – Envoyer le token depuis le client (JavaScript)

5 Gestion des Erreurs

5.1 Middleware d'erreur

Un middleware d'erreur a 4 paramètres : (err, req, res, next).

```
1 // Declarer APRES toutes les routes
2 const errorHandler = (err, req, res, next) => {
3   console.error('ERREUR:', err.message);
4
5   const statusCode = err.statusCode || 500;
6
7   res.status(statusCode).json({
8     success: false,
9     error: err.message || 'Erreur serveur',
10    stack: process.env.NODE_ENV === 'development' ? err.stack
11    : undefined
12  });
13
14 app.use(errorHandler);
```

Listing 17 – Middleware de gestion d'erreurs

5.2 Déclencher une erreur

```
1 app.get('/api/users/:id', async (req, res, next) => {
2   try {
3     const user = await User.findById(req.params.id);
4
5     if (!user) {
6       const error = new Error('Utilisateur non trouve');
7       error.statusCode = 404;
8       return next(error); // Passe au middleware d'erreur
9     }
10
11    res.json(user);
12  } catch (error) {
13    next(error);
14  }
15 });
```

Listing 18 – Passer une erreur au middleware

5.3 Middleware 404

```
1 // APRES toutes les routes, AVANT errorHandler
2 app.use((req, res) => {
3   res.status(404).json({
4     error: 'Route non trouvee',
5     path: req.originalUrl
6   });
7 });
```

```
7 });
```

Listing 19 – Gérer les routes non trouvées

6 Ordre d'Exécution

L'ordre de déclaration des middlewares est **crucial**.

✖ Erreur courante

```
1 // ERREUR: express.json() declare APRES la route
2 app.post('/api/users', (req, res) => {
3     console.log(req.body); // undefined!
4 });
5 app.use(express.json()); // Trop tard!
```

✔ Ordre recommandé

1. Middlewares tiers (cors, morgan)
2. Middlewares de parsing (express.json, urlencoded)
3. Fichiers statiques (express.static)
4. Middlewares personnalisés (logger)
5. Routes de l'application
6. Middleware 404
7. Middleware d'erreur (toujours en dernier)

7 Middlewares Tiers

7.1 CORS

Autorise les requêtes depuis d'autres domaines (frontend/backend séparés).

```
1 npm install cors

1 const cors = require('cors');
2
3 // Autoriser toutes les origines (developpement)
4 app.use(cors());
5
6 // Configuration specifique (production)
7 app.use(cors({
8   origin: 'https://monsite.com',
9   methods: ['GET', 'POST', 'PUT', 'DELETE'],
10  credentials: true
11 }));
```

Listing 20 – Configuration CORS

7.2 Morgan

Logger HTTP professionnel.

```
1 npm install morgan

1 const morgan = require('morgan');
2
3 app.use(morgan('dev')); // Format colore pour developpement
4 // GET /api/users 200 15.234 ms - 523
5
6 app.use(morgan('combined')); // Format Apache pour production
```

Listing 21 – Utilisation de Morgan

7.3 Autres middlewares utiles

Package	Usage
helmet	Sécurise l'app en ajoutant des headers HTTP
compression	Compresse les réponses pour réduire la bande passante
express-rate-limit	Limite le nombre de requêtes (protection DDoS)
express-validator	Validation avancée des données
multer	Upload de fichiers

8 Application Complète

```
1 const express = require('express');
2 const mongoose = require('mongoose');
3 const cors = require('cors');
4 const morgan = require('morgan');
5 const jwt = require('jsonwebtoken');
6 const bcrypt = require('bcryptjs');
7
8 const app = express();
9 const JWT_SECRET = process.env.JWT_SECRET || 'secret_dev_123';
10
11 // === === === MIDDLEWARES === === ===
12 app.use(cors());
13 app.use(morgan('dev'));
14 app.use(express.json());
15
16 // === === === MIDDLEWARE AUTH === === ===
17 const auth = (req, res, next) => {
18   const authHeader = req.headers['authorization'];
19   if (!authHeader) return res.status(401).json({ error: 'Token
   requis' });
20
21   const token = authHeader.split(' ')[1];
22   try {
23     req.user = jwt.verify(token, JWT_SECRET);
24     next();
25   } catch (err) {
26     res.status(401).json({ error: 'Token invalide' });
27   }
28 };
29
30 // === === === ROUTES PUBLIQUES === === ===
31 app.post('/api/auth/register', async (req, res) => {
32   const { nom, email, password } = req.body;
33   const hashedPassword = await bcrypt.hash(password, 10);
34   // Sauvegarder en BDD...
35   const token = jwt.sign({ email }, JWT_SECRET, { expiresIn:
   '24h' });
36   res.status(201).json({ token });
37 });
38
39 app.post('/api/auth/login', async (req, res) => {
40   // Verifier credentials et generer token...
41   const token = jwt.sign({ email: req.body.email }, JWT_SECRET,
   { expiresIn: '24h' });
42   res.json({ token });
43 });
44
45 // === === === ROUTES PROTEGEES === === ===
46 app.get('/api/profile', auth, (req, res) => {
47   res.json({ user: req.user });
48 });
```

```
49
50 // === === === = ERROR HANDLING === === === =
51 app.use((req, res) => {
52   res.status(404).json({ error: 'Route non trouvee' });
53 });
54
55 app.use((err, req, res, next) => {
56   res.status(500).json({ error: err.message });
57 });
58
59 // === === === = DEMARRAGE === === === =
60 mongoose.connect('mongodb://localhost:27017/myapp')
61   .then(() => {
62     app.listen(3000, () => console.log('Serveur sur port
63     3000'));
64   });
```

Listing 22 – API Express avec JWT - index.js

9 Synthèse

Points clés

1. **Middleware** = fonction (req, res, next)
2. **next()** passe au middleware suivant
3. **JWT** = Token stateless pour l'authentification
4. Structure JWT : Header.Payload.Signature
5. Middleware d'erreur = **4 paramètres** (err, req, res, next)
6. L'**ordre** de déclaration est crucial

9.1 Récapitulatif JWT

Action	Méthode
Créer un token	jwt.sign(payload, secret, options)
Vérifier un token	jwt.verify(token, secret)
Header HTTP	Authorization: Bearer <token>

9.2 Template Express + JWT

```

1 // 1. Middlewares tiers
2 app.use(cors());
3 app.use(morgan('dev'));
4
5 // 2. Parsing
6 app.use(express.json());
7
8 // 3. Routes publiques (auth)
9 app.use('/api/auth', authRoutes);
10
11 // 4. Routes protegees
12 app.use('/api/users', authMiddleware, userRoutes);
13 app.use('/api/products', authMiddleware, productRoutes);
14
15 // 5. 404 handler
16 app.use(notFoundHandler);
17
18 // 6. Error handler
19 app.use(errorHandler);

```

Listing 23 – Structure recommandée